Experiment 11: 3D Convection–Diffusion (Burgers-type) Equations

Ayush Singh

November 4, 2026

1 Aim

Solve the 3D coupled nonlinear convection–diffusion (viscous Burgers) equations on a cubic domain with the Finite Difference Method and an explicit time-marching scheme, and study how an initial high-velocity cube diffuses and convects until convergence.

2 Assumptions & Limitations

- **Unsteady, 3D, Nonlinear:** The velocity components (u, v, w) transport themselves (Burgers-type convection).
- **No Pressure, No Body Force:** Only convection and diffusion are modelled; incompressibility is not enforced.
- **Constant Viscosity:** $\nu = 0.3$ is uniform.
- **Uniform Cubic Grid:** $\Delta x = \Delta y = \Delta z$ and $n_x = n_y = n_z = 21$.
- **Explicit First-order Scheme:** Forward Euler in time and forward (first-order) differencing for convection, so accuracy is first order and stability is conditional. Stability is aided by the diffusion-dominated regime (cell Péclet number $u\Delta x/\nu < 2$).
- **Fixed Dirichlet Boundaries:** All outer faces are held at the ambient value.

3 Governing Equations & Simplification

The 3D viscous Burgers system for $\mathbf{u} = (u, v, w)$ is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} + w\frac{\partial u}{\partial z} = \nu\nabla^2 u \tag{1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} + w\frac{\partial v}{\partial z} = \nu\nabla^2 v \tag{2}$$

$$\frac{\partial w}{\partial t} + u\frac{\partial w}{\partial x} + v\frac{\partial w}{\partial y} + w\frac{\partial w}{\partial z} = \nu\nabla^2 w \tag{3}$$

with $\nabla^2 = \partial_x^2 + \partial_y^2 + \partial_z^2$. These are the Navier–Stokes momentum equations with the pressure-gradient term removed, which isolates the nonlinear convection and the viscous diffusion and makes the problem a convenient 3D test of the discretization. The velocity magnitude plotted is $|\mathbf{V}| = \sqrt{u^2 + v^2 + w^2}$.

4 Discretization & Algorithm

4.1 Finite-Difference Scheme

With $\Delta x = \Delta y = \Delta z$, the update for interior nodes (i, j, k) is

$$u_{i,j,k}^{n+1} = u_{i,j,k}^{n} + \frac{\Delta t}{\Delta x^2}\Big[\nu\big(u_{i+1} + u_{i-1} + u_{j+1} + u_{j-1} + u_{k+1} + u_{k-1} - 6u_{i,j,k}\big) - \Delta x\big(u\,(u_{i+1} - u_{i,j,k}) + v\,(u_{j+1} - u_{i,j,k}) + w\,(u_{k+1} - u_{i,j,k})\big)\Big]^n \quad (4)$$

and identically for v and w, where the subscripts $j \pm 1$, $k \pm 1$ denote neighbours in y and z. Simplifying the factor $\Delta t/\Delta x^2 \cdot \Delta x$ shows the convective term is

$$-\frac{\Delta t}{\Delta x}\left[u\,\delta_x u + v\,\delta_y u + w\,\delta_z u\right], \qquad \delta_x u = u_{i+1} - u_i \quad (5)$$

i.e. a first-order forward-difference convection, and the diffusive term is the standard 7-point (second-order central) Laplacian. All three components are updated from the *old* fields (u^n, v^n, w^n), so the coupling is explicit.

4.2 Time Step and Stability

The time step is set through a diffusion-based Courant-type number:

$$\Delta t = C\frac{\Delta x^2}{\nu}, \qquad C = 0.1 \quad (6)$$

For the explicit 3D diffusion update to be stable, $\nu\Delta t/\Delta x^2 \le 1/6$; here this ratio equals $C = 0.1$, which satisfies the condition. The convective Courant number $|u|\Delta t/\Delta x$ is also small.

4.3 Algorithm

1. Initialize $u = v = w = 1$ everywhere and set the cube $0.5 \le x, y, z \le 1.0$ to 2 in all three components.

2. Copy the current fields; update interior nodes with the scheme above.

3. Re-impose the boundary values on all six faces.

4. Compute the convergence measure $\|u - u^n\| + \|v - v^n\| + \|w - w^n\|$; stop when it falls below 10^{-5} (or after the maximum number of frames).

5 Code Structure & Meaning of Comments

- `nx = ny = nz` and `dx = dy = dz` – uniform cubic mesh.
- `dt = courant * dx**2 / nu` – diffusion-limited explicit time step.
- `tolerance = 1e-5 (threshold for convergence)` – stopping criterion on field change per step.
- `Initial and Boundary Condition` – the high-velocity cube inside an ambient field of 1.
- `Function to update the field` – `conv_diffuse` advances u, v, w one step using NumPy slicing over interior nodes (no explicit loops).
- `Boundary Conditions: Bottom/Top/Left/Right/Back/Front face` – Dirichlet values for each of the six faces.
- `zvalue` and the commented contour code – optional 2D slice view; the 3D scatter version is used for the final results.

6 Methodology and Boundary Conditions

Parameter	**Value**	**Parameter**	**Value**
Domain length l	2.0	Grid ($n_x = n_y = n_z$)	21
Grid spacing Δx	0.1	Viscosity ν	0.3
Courant number C	0.1	Time step Δt	$\approx 3.3 \times 10^{-3}$
Ambient velocity	1	Initial cube velocity	2
Convergence tolerance	10^{-5}	Frames (max steps)	200

Table 1: Parameter set for the 3D convection–diffusion simulation.

- **Initial condition:** $u = v = w = 2$ inside the cube $[0.5, 1.0]^3$, and 1 elsewhere.
- **Boundary conditions:** Dirichlet, $u = v = w = 1$ on all six faces of the domain.

7 Results & Discussions

Figure 1 shows the initial 3D scatter plot of the velocity magnitude, with a compact region of high speed (the cube) embedded in the uniform ambient field.

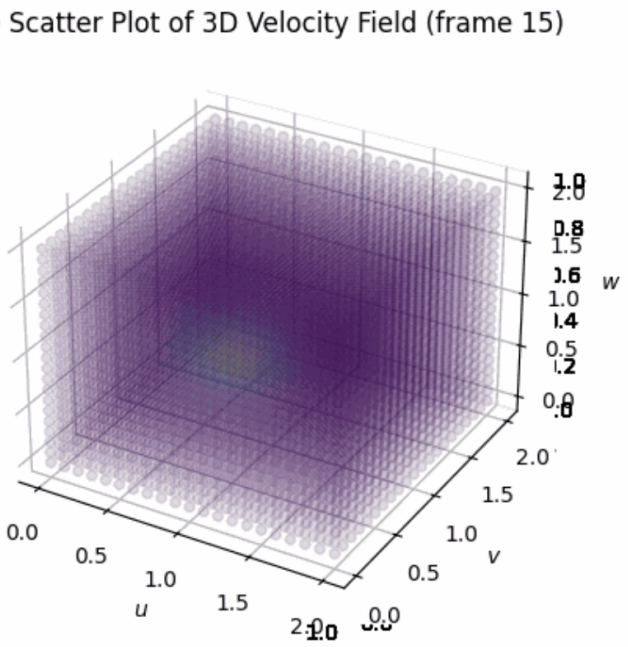


Figure 1: 3D scatter plot of velocity magnitude initially.

Figure 2 shows the field after the iterations. The sharp-edged cube has spread out: viscous diffusion smooths the gradients at its edges and lowers its peak value, while the nonlinear convective term shifts and distorts the disturbance. The values relax towards the boundary value of 1, as dictated by the Dirichlet boundaries.

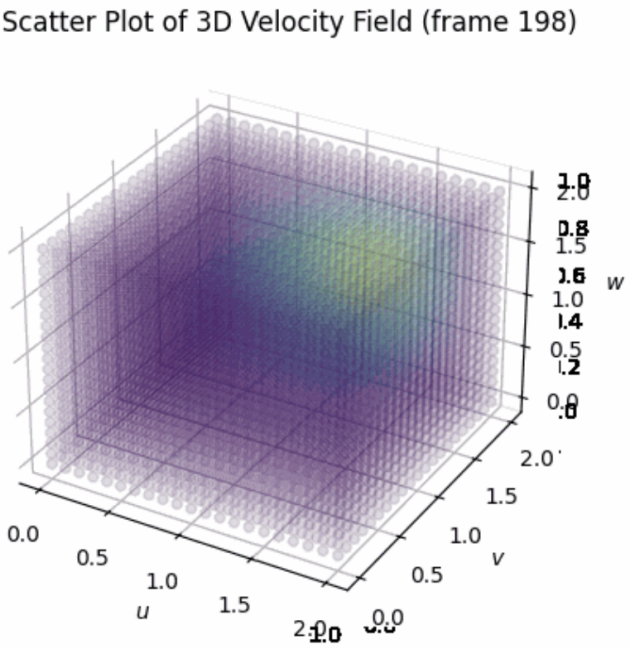


Figure 2: 3D scatter plot of velocity magnitude after the simulation.

8 Conclusion

The 3D coupled convection–diffusion equations were solved with an explicit finite-difference scheme using vectorized slicing. The diffusion-controlled time step $\Delta t = 0.1\Delta x^2/\nu$ ensured stable marching, and the initial high-velocity cube diffused towards the ambient state. The scheme is only first-order accurate in convection and time; upwinding (according to velocity sign), higher-order schemes or implicit solvers would be needed for convection-dominated cases or larger time steps.